

Assignment 1: Deploy All Environments

As part of the assignment, four runtime environments for the k-NN application were deployed: AWS Lambda (zip), AWS Lambda (container), ECS Fargate, and EC2. Then, test requests were sent to all endpoints using the same query vector.

The comparison of the responses showed that each environment returned an identical results array containing the same indices of the nearest neighbors and the same distance values. This means that in all cases the same application logic was executed and the same input dataset was used.

Assignment 2: Scenario A — Cold Start Characterization

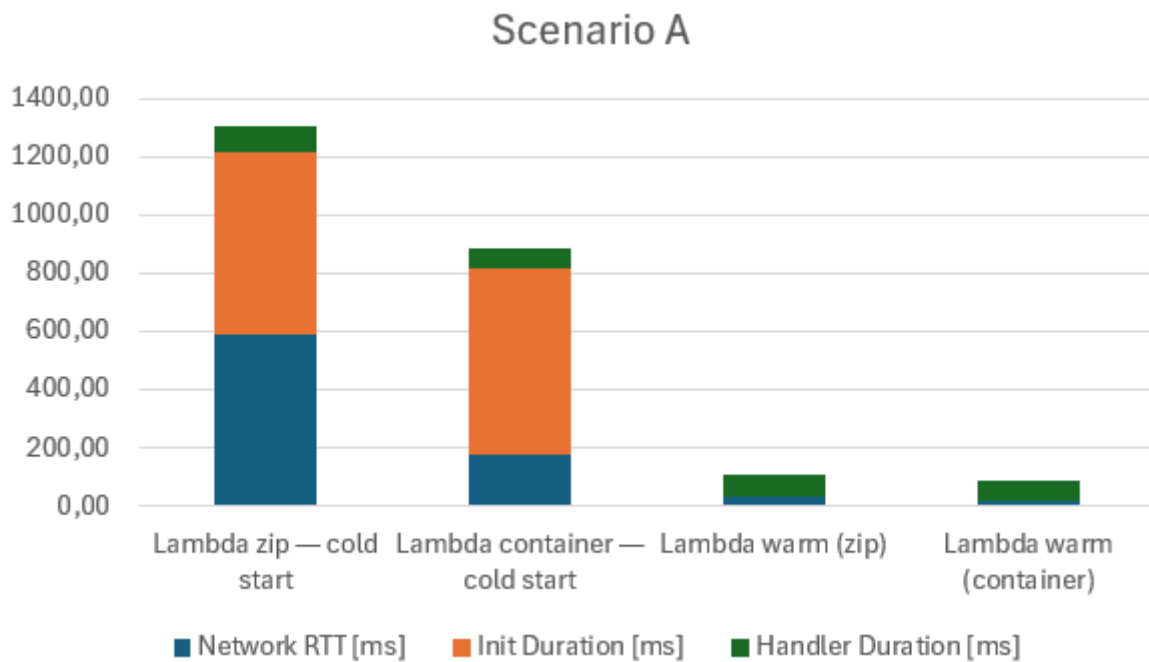
Variant	Requests	Slowest	Fastest	Average	p50	p95	p99
Lambda zip	30	1302.4 ms	83.6 ms	147.1 ms	108.4 ms	140.5 ms	1302.4 ms
Lambda container	30	886.7 ms	78.4 ms	119.4 ms	88.6 ms	148.2 ms	886.7 ms

The response-time histograms show one clearly visible high-latency outlier for each Lambda variant. This is consistent with a cold-start request, while the remaining requests form the warm-invocation cluster.

CloudWatch REPORT lines confirm cold starts through the presence of Init Duration.

Case	Total client latency [ms]	Init Duration [ms]	Handler Duration [ms]	Estimated Network RTT [ms]
Lambda zip — cold start	1302.40	627.60	87.50	587.30
Lambda container — cold start	886.70	640.74	72.16	173.80
Lambda warm (zip, p50-based)	108.40	0.00	77.63	30.77
Lambda warm (container, p50-based)	88.62	0.00	72.70	15.92

Zip deployments are usually slightly faster on cold start, because the package is smaller and simpler, while container-based Lambda may pay additional image-related initialization overhead.



Assignment 3: Scenario B — Warm Steady-State Throughput

Environment	Concurrency	p50 (ms)	p95 (ms)	p99 (ms)	p99 > 2×p95?
Lambda (zip)	5	95.89	112.88	139.62	No
Lambda (zip)	10	94.35	114.47	160.09	No
Lambda (container)	5	95.43	111.39	172.57	No
Lambda (container)	10	91.59	109.03	146.63	No
Fargate	10	802.80	1011.40	1111.00	No
Fargate	50	4021.00	4704.20	4815.90	No
EC2	10	199.30	255.71	297.62	No
EC2	50	913.30	1051.90	1103.90	No

The results show a clear difference between Lambda and the always-on environments. For both Lambda variants, p50 changes only slightly when concurrency increases from 5 to 10, which means warm requests keep similar latency under higher parallel load. This is expected, because Lambda handles concurrent requests using separate execution environments rather than forcing them to wait in one shared queue.

In contrast, Fargate and EC2 become much slower as concurrency increases. Their requests are handled by a single task or instance, so higher concurrency leads to queuing and CPU contention. This is especially visible for Fargate, where p50 increases from 802.80 ms at concurrency 10 to 4021.00 ms at concurrency 50. EC2 shows the same trend, but less severely.

No row satisfies the condition $p99 > 2 \times p95$, so there is no clear tail-latency instability by the assignment criterion. Overall, Lambda achieved the best warm steady-state latency, EC2 was intermediate, and Fargate

performed worst in the tested configuration.

Assignment 4: Scenario C — Burst from Zero

This experiment simulated a burst of 200 requests after 20 minutes of inactivity.

Lambda was tested at concurrency = 10, while Fargate and EC2 were tested at concurrency = 50, as required by the assignment.

Environment	Concurrency	p50 (ms)	p95 (ms)	p99 (ms)	Max latency (ms)	Cold starts
Lambda (zip)	10	95.2	648.3	1149.5	1195.7	10
Lambda (container)	10	97.3	459.4	1527.1	1576.3	10
EC2	50	866.6	1059.4	1105.9	1110.3	0
Fargate	50	3893.5	4176.2	4294.1	4299.4	0

The burst test shows a clear difference between Lambda and the always-on environments.

For both Lambda variants, p50 remains low (around 96–98 ms), but p95 and p99 increase sharply because the burst triggers cold starts after the idle period. CloudWatch logs confirmed 10 cold starts for each Lambda variant, which matches the concurrency limit of 10.

This creates a bimodal latency pattern in Lambda: one cluster of fast warm requests and another cluster of much slower cold-start requests. As a result, Lambda has a low median latency but a much higher tail latency during burst traffic.

Neither Lambda variant meets the p99 < 500 ms SLO in this scenario:

- Lambda (zip): p99 = 1149.5 ms
- Lambda (container): p99 = 1527.1 ms

To meet the SLO under burst traffic, Lambda would require changes such as provisioned concurrency so that pre-initialized execution environments are already available when the burst arrives.

EC2 does not suffer from cold starts, so its latency distribution is more uniform, although still higher than Lambda's warm requests because all burst traffic is handled by a single always-on instance. Fargate performs worst here, with very high p50, p95, and p99, showing strong queuing and CPU contention under burst load.

Assignment 5: Cost at Zero Load

The lab configurations were:

- **Lambda:** 512 MB function
- **Fargate:** 0.5 vCPU, 1 GB RAM
- **EC2:** t3.small

AWS Lambda is billed based on the **number of requests** and the **execution duration**, so when the function is not invoked, its idle cost is zero. :contentReference[oaicite:7]{index=7}

For AWS Fargate in **US East (N. Virginia)**, the Linux/x86 rates are **\$0.000011244 per vCPU-second** and **\$0.000001235 per GB-second**. :contentReference[oaicite:8]{index=8}

For Amazon EC2, the **t3.small** Linux on-demand hourly price in **US East (N. Virginia)** is **\$0.0209/hour**.

Idle cost table

Environment	Lab configuration	Hourly idle cost (USD)	Monthly idle cost for 18 h/day idle (USD)	Full 24/7 monthly cost (USD)
Lambda	512 MB	0.0000	0.0000	0.0000
Fargate	0.5 vCPU, 1 GB RAM	0.0247	13.3290	17.7733
EC2	t3.small	0.0209	11.2860	15.0480

The only environment with **zero idle cost** is **Lambda**, because it is billed only when requests are executed. In contrast, **Fargate** and **EC2** are always-on environments, so they continue to generate cost even when they are not serving any traffic.

Under the assumption of **18 hours per day idle** and **6 hours per day active**, the idle portion alone is already significant for always-on deployments:

- **Fargate:** about **\$13.33/month**
- **EC2:** about **\$11.29/month**
- **Lambda:** **\$0/month**

This shows why Lambda is financially attractive for workloads with long idle periods and bursty traffic.

Assignment 6: Cost Model, Break-Even, and Recommendation

Environment	Estimated monthly cost (USD)	Meets p99 < 500 ms SLO?
Lambda (zip)	7.16	No
Lambda (container)	6.75	No
EC2	15.05	No
Fargate	17.77	No

Under the given traffic model, Lambda is the cheapest option because it has zero cost during idle periods and the average load is only about **3.23 RPS**. The estimated monthly costs are about **\$7.16** for Lambda zip, **\$6.75** for Lambda container, **\$15.05** for EC2, and **\$17.77** for Fargate. The Lambda/Fargate break-even point is around **8–8.5 RPS**, so this workload is clearly below that level.

However, cost is not enough, because the target SLO is **p99 < 500 ms**. Based on Scenario C, none of the environments meets this SLO as deployed. Lambda fails because burst traffic after idle causes cold starts, while EC2 and Fargate fail because a single instance/task becomes a bottleneck under high concurrency. Therefore, the best practical recommendation is **Lambda with Provisioned Concurrency** during the active

window: default Lambda is the cheapest, but Provisioned Concurrency would be needed to reduce burst latency and make the solution closer to the required SLO.